

dataQ 블랙박스 테스트: 83%에서 97.6%까지 간 세 라운드

Phase 2가 끝나고 Phase 3로 넘어가는 길목에서 블랙박스 테스트를 한 번 제대로 돌릴 필요가 있었다. Phase 1·2에 걸쳐 기능이 빠르게 쌓인 뒤라 단위 테스트(229건, 전부 PASS)만으로는 커버가 안 되는 구간이 많았다. 특히 화면 동선·권한·실제 DBMS 접속처럼 단위 테스트가 닿기 어려운 부분이 의심스러웠다.

Selenium + Edge WebDriver(headless)와 requests API를 섞어서 104건짜리 시나리오 세트를 만들었다. 인증·대시보드·표준사전 CRUD·진단·구조 진단·자동 표준화·승인·검색·ERwin 임포트까지 16개 카테고리. 첫 실행은 2026-04-02 이른 오후였다.

1라운드: 83.1%, 그리고 "왜 이렇게 낮지"

첫 결과는 69/83 PASS. 104건 중 21건이 SKIP(아직 기능이 없거나 테스트 데이터가 부족해서 건너뛴 것)이었고 실질 대상은 83건이었다. 83.1%는 "기능이 돌긴 도는데 가장자리에서 튕다"는 신호에 가까운 숫자였다. 시나리오 하나하나를 열어보면서 FAIL의 원인을 네 가지 부류로 나눴다.

첫째, **순수 버그**. 예를 들어 대시보드의 준수율 쿼리에 GROUP BY가 빠져서 PostgreSQL이 "집계와 비집계 컬럼이 섞였다"고 던지던 건. 단일 모델 환경에서는 우연히 돌아가던 게 여러 모델 중 하나를 선택하는 시나리오에서 터졌다. 한 줄짜리 SQL 수정.

둘째, **테스트가 잘못 짠 것**. 일괄 삭제 시나리오에서 기대값을 "빈 목록"으로 둔 건이 있었는데, 실제로는 삭제 확인 모달이 뜨고 확인을 눌러야 진행되는 구조였다. 테스트가 모달을 닫는 동작을 안 넣었던 것. Selenium이 '삭제' 버튼을 눌렀다고 해서 삭제가 된 건 아니었다.

셋째, **데이터 의존**. 특정 시나리오가 "단어 3건 이상"을 전제로 하는데 테스트 시작 시점 DB에 2건만 있으면 실패. 실행 전 seed 스크립트로 최소 데이터를 보장하도록 고쳤다.

넷째, **진짜 기능 공백**. 빠른 액션의 "구조 진단으로 바로 가기" 버튼이 라우터 경로를 잘못 가리키고 있어 화면이 열리지 않던 건. 사용 빈도가 낮아서 발견이 늦었다.

이 넷 중 첫째와 넷째는 코드 수정, 둘째와 셋째는 테스트 쪽 수정이었다. 버그 4건을 실제로 고친 뒤 재실행한 게 다음 라운드다.

2라운드: 92.8%, 그리고 "왜 더 오르지 않지"

두 번째 실행 결과는 77/83, 92.8%. 버그 4건을 고쳤고 새 테스트 케이스도 몇 건 추가했다. 83% → 92.8%는 예상한 범위 안이었지만, 두 가지가 이상했다.

하나는 같은 카테고리에서 이번에 새로 FAIL로 떨어진 건이 있었다는 것. 지난 라운드에는 PASS였다. 추적해 보니 1라운드에서 만든 **테스트 잔재 데이터**가 문제였다. 예를 들어 "단어 신규 등록 → 등록 확인" 시나리오가 `TSTWRD` 라는 단어를 만들고 끝난다. 2라운드에서 같은 시나리오가 또 돌면 `TSTWRD` 가 이미

존재해서 "중복 영문약어" 에러로 실패한다. 1라운드는 DB가 깨끗했기 때문에 PASS였고, 2라운드는 1라운드의 잔재 때문에 FAIL이었다.

다른 하나는 "일괄 등록" 시나리오가 계속 Flaky하다는 것. 실행할 때마다 결과가 바뀌었다. 로그를 뒤져보니 일괄 등록이 비동기로 도는 구간이 있어서 테스트가 결과를 확인하는 시점과 실제 완료 시점이 어긋날 때가 있었다. `time.sleep(2)` 를 고정으로 넣는 건 나쁜 선택이다(환경마다 빠르고 느림) — 그래서 완료 상태를 짧게 폴링하는 헬퍼를 만들어서 바꿨다.

테스트 코드에 "정리 로직"이라는 이름으로 한 덩어리를 추가했다. 각 시나리오가 생성한 테스트 데이터에 공통 prefix(`_BB_`)를 붙여두고, 실행 전/후에 이 prefix를 가진 건을 전부 DELETE한다. 시나리오 간 격리. 이건 2라운드가 아니라 2라운드에서 발견된 문제를 3라운드에 반영한 것.

3라운드: 93.9%, 절반만 풀린 속제

정리 로직을 넣고 다시 돌린 결과는 78/83, 93.9%. 92.8%에서 93.9%로 올랐다. 딱 1건 플러스. 기대했던 것보다는 덜 오른 수치였다.

이유는 두 가지였다. 정리 로직이 "테스트가 만든 데이터"만 정리하고 "테스트가 실패해서 중간 상태로 남은 데이터"는 정리하지 못했다. 실패한 시나리오가 남긴 부분 데이터가 다음 라운드에서 또 다른 실패를 유발하는 체인이 있었다. prefix 기준 정리에 "테스트 시작 전에 prefix로 남아 있는 것 전부 지우기"를 추가해서 매 라운드가 완전히 깨끗한 상태에서 시작하도록 했다.

그리고 2-10번 "빠른 액션 — 구조 진단" 건은 여전히 FAIL이었다. 이건 버그 4건 수정에서 놓친 건이었다. 증상은 `error:Cannot read properties of undefined (reading '_instance')` — Vuetify 탭 컴포넌트 참조가 빈 경우에 터지는 전형적인 런타임 에러였다. 2라운드 때는 같은 에러가 다른 빠른 액션 3건(표준화 추천/진단 실행/구조 진단)에서 나타났는데, 이번엔 그 중 두 건은 해결됐고 구조 진단 한 건만 남아 있었다. 해당 탭 초기화 순서 문제였다.

4라운드: 97.6%, 두 건의 잔존 FAIL

마지막 실행은 81/83, 97.6%. PASS 81, FAIL 2, SKIP 21. FAIL 2건은 다음과 같았다.

- **2-10 빠른 액션 구조 진단:** 앞에서 말한 Vuetify 탭 초기화 순서. 화면은 뜨는데 특정 상황에서 이벤트 버스가 늦게 바인딩되어 터진다. 재현율이 낮고 수정 범위가 라우터·대시보드·탭 전반에 걸쳐서 이 라운드에서는 남겨두기로 했다. 블랙박스 테스트의 "이 라운드 안에 해결 가능한 것만 잡는다"는 기준에 걸린 것.
- **4-3 용어 한 건:** 특정 입력(영문약어에 특수문자가 섞인 케이스)에서 등록이 실패하는 건. 검증 로직이 먼저 걸려야 하는데 DB까지 내려가서 제약조건 위반 에러를 던지고 있었다. 프론트 검증 + 백엔드 검증 둘 다 강화해야 하는 건이었고, 이것도 이 라운드에서는 남겼다.

두 건을 남긴 이유는 단순하다. "97.6%까지 올렸고 남은 2건은 구조적 수정이 필요하다" — 다음 사이클로 넘기는 게 맞는 상황이었기 때문이다. 블랙박스 테스트는 숫자가 100%가 되는 것 자체가 목표가 아니라, "현재 시점에 무엇이 튀는가"를 한 장으로 보는 게 목표다. 97.6%는 그 장이 거의 깨끗하다는 신호였다.

테스트 라운드에서 얻은 것

세 라운드를 돌리면서 실제로 바뀐 건 코드보다 테스트 인프라 쪽이었다. 버그 수정은 4건이지만, 테스트 쪽에서는 정리 로직, 폴링 헬퍼, prefix 격리, 실행 전 seed 보장, 실패 시나리오 복원 경로까지 다섯 가지가 붙었다. 다음에 이 블랙박스 세트를 돌릴 때는 이 다섯 가지가 기본으로 들어간 상태에서 시작한다.

반대로 말하면, 처음 만든 블랙박스 세트는 이 다섯 가지가 없었기 때문에 "진짜 실패한 건"과 "테스트 환경이 만들어낸 실패"를 섞어서 보고했다는 뜻이다. 1라운드 83.1%의 상당 부분은 후자였다. 103건을 짜는 건 비교적 쉽고, 그 103건이 반복 실행에서 일관된 결과를 주게 만드는 건 별개의 작업이었다. 이것 미리 알았더라면 테스트 세트를 짤 때 첫날부터 정리 로직을 같이 넣었을 거다.